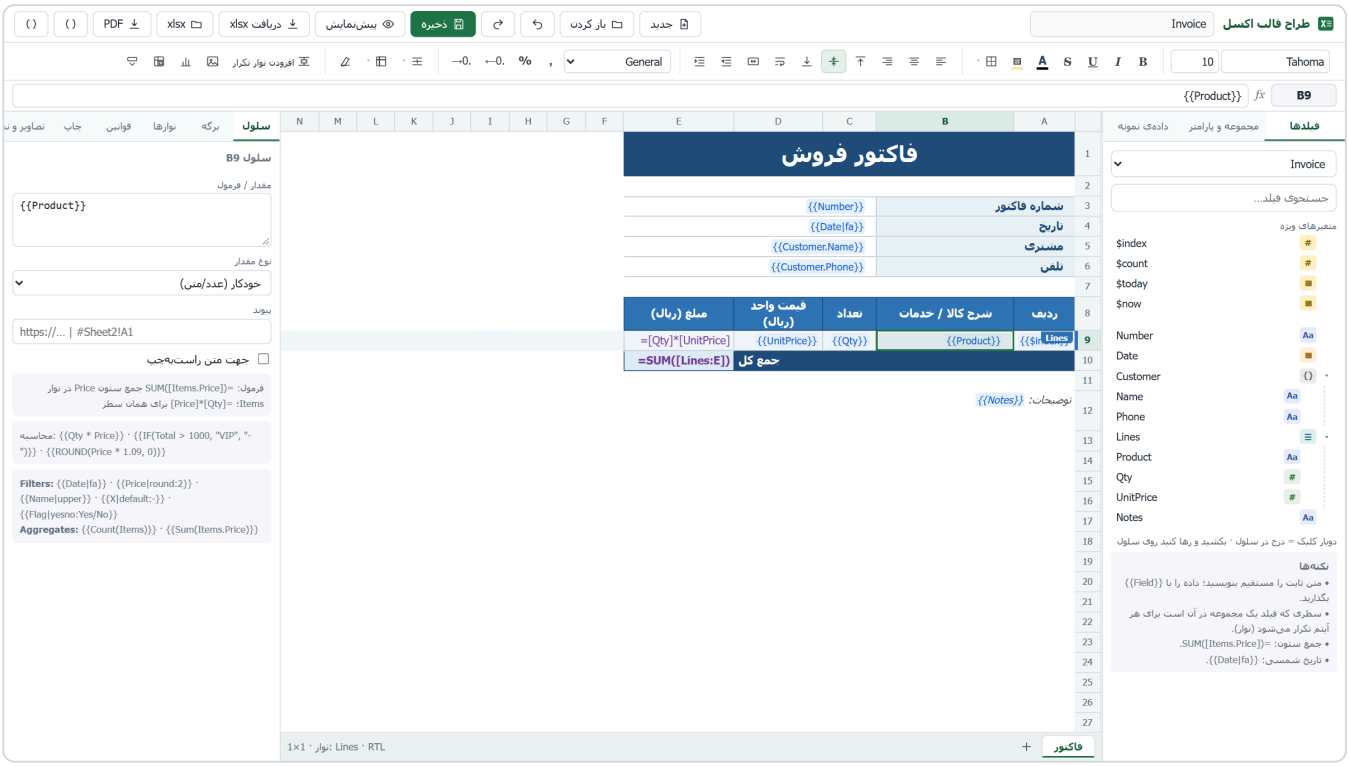
راهنمای کامل KSC.Excel

طراح تصویری قالب اکسل و PDF + موتور تولید گزارش برای .NET و .NET Framework.

قالب را یک‌بار در یک طراح تصویری می‌کشید و می‌سازید؛ برنامه‌نویس فقط داده را می‌دهد و فایل `.xlsx` (یا PDF) زیبا و کامل می‌گیرد — بدون درگیر شدن با OpenXML SDK / ClosedXML / EPPlus، بدون نصب Excel روی سرور، و بدون هیچ CDN (مناسب شبکه‌ی داخلی).

```
return this.ExcelFile("Invoice", model, "invoice.xlsx");     // Excel
return File(svc.RenderPdf("Invoice", model), "application/pdf", "invoice.pdf");   // PDF
```



> نسخه‌ی انگلیسی: KSC.Excel-Guide.en.md · سند طراحی و معماری: ../KSC.Excel-Design.md

فهرست

1. این ابزار چیست و چه می‌دهد؟
2. شروع سریع: طراح را چطور باز کنم؟
3. نصب و پیش‌نیازها
4. تور طراح
5. آموزش گام‌به‌گام: ساخت یک فاکتور
6. زبان قالب
7. راهنمای برنامه‌نویس — .NET (8/10)

8. راهنمای برنامه‌نویس — .NET Framework
9. برنامه‌ی ویندوزی طراح
10. PDF
11. گزارش‌های بزرگ (Streaming)
12. منبع داده‌ی SQL، پارامتر، توابع سفارشی
13. ورود از فایل Excel
14. مرجع تنظیمات و API
15. امنیت
16. استقرار
17. کارایی و محدودیت‌ها
18. عیب‌یابی و پرسش‌های متداول
19. ساخت از سورس، تست و مشارکت
20. نقشه‌ی راه و محدودیت‌های شناخته‌شده

---

۱) این ابزار چیست و چه می‌دهد؟

مسئله

تولید فایل Excel زیبا در برنامه‌های سازمانی یعنی انتخاب یک کتابخانه، نوشتن ده‌ها خط کد قالب‌بندی برای هر گزارش و تکرار آن با هر تغییر ظاهری. تغییر یک رنگ = Build و Deploy.

راه‌حل

بخش	توضیح
طراح تصویری	شبکه‌ی سلولی شبیه Excel با پنل داده، نوار تکرار، قالب شرطی، نمودار، پیش‌نمایش؛ فارسی و راست‌به‌چپ
موتور	قالب JSON + داده ← `.xlsx` / PDF؛ بدون وابستگی به کتابخانه‌ی Excel
میزبان‌ها	ASP.NET Core (8/10)، ASP.NET MVC 5 (.NET Framework 4.7.2+)، برنامه‌ی ویندوزی
نمایش	در صفحه (`KscExcelViewer` :HTML)، **نمایشگر PDF در صفحه** (`KscPdfViewer` / `Html.KscPdfViewer`) یا دانلود

بسته‌ها

بسته	Target	کاربرد
`KSC.Excel`	`netstandard2.0`	هسته: مدل قالب، موتور رندر، نویسنده‌ی XLSX، ورود xlsx، پیش‌نمایش HTML، API و UI طراح (Embedded)

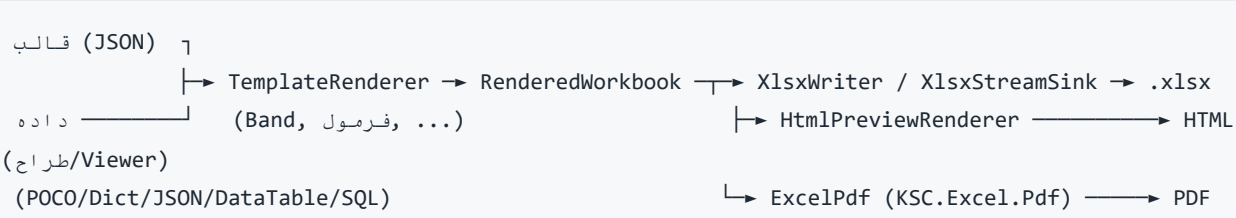
بسته	Target	کاربرد
`KSC.Excel.AspNetCore`	`net8.0`, `net10.0`	`AddKscExcel`، `MapKscExcelDesigner`، `ExcelFile`، `KscExcelViewer`
`KSC.Excel.Mvc`	`net472`, `net48`	`KscExcel.Initialize`، مسیر طراح، `ExcelResult`، `Html.KscExcelViewer`
`KSC.Excel.Pdf`	`netstandard2.0`	خروجی PDF با شکل‌دهی درست فارسی (SkiaSharp + HarfBuzz)؛ روی .NET Framework 4.6.1+ **و** .NET 8/10
`KSC.Excel.Designer.Windows`	`net10.0-windows`, `net48`	برنامه‌ی ویندوزی طراح (WPF + WebView2)؛ یک کد، دو runtime

تنها وابستگی هسته: `Newtonsoft.Json`.

امکانات کلیدی

- **داده:** `{{Field}}`، عبارت محاسباتی `{{Qty * Price}}`، فیلتر (`|fa` تاریخ شمسی، `|round`، ...)، توابع تجمیعی، متغیرهای ویژه.
- **ساختار:** نوار تکرار (Band) و تو در تو، مرتب‌سازی و گروه‌بندی با جمع هر گروه، **جدول متقاطع (Cross-tab)** با ستون‌های پویا، تکرار برگه، بخش‌های گزارش (سرصفحه/سرگزارش/پاگزارش)، **ویزارد گروه‌بندی**.
- **فایل:** فرمول زنده (با مقدار ذخیره‌شده)، ادغام، فریز، فیلتر، قالب شرطی (۷ نوع)، اعتبارسنجی، تصویر (ثابت/پویا)، **نمودار** (ستونی، میله‌ای، خطی، ناحیه‌ای، دایره‌ای)، لینک، تنظیمات چاپ.
- **داده‌ی بیرونی: SQL** (فقط‌خواندنی، پارامتری)، پارامتر ورودی، تابع/فیلتر سفارشی.
- **مقیاس:** نویسنده‌ی **Streaming** (۱ میلیون سطر ≈ ۱۲ ثانیه، ≈ ۴۷ مگابایت حافظه).
- **سایر: ورود از xlsx موجود**، خروجی **PDF**، برنامه‌ی ویندوزی.
- **کیفیت:** خروجی در Excel واقعی باز و بررسی شده؛ تست‌ها روی .NET 10 و .NET Framework 4.8.

معماری یک‌نگاه



---

۲) شروع سریع: طراح را چطور باز کنم؟

طراح یک **صفحه‌ی وب** است که داخل برنامه‌ی خودتان (یا در پنجره‌ی ویندوزی) باز می‌شود. چهار راه:

الف) اجرای نمونه‌ی آماده (سریع‌ترین راه)

```
git clone <آدرس-ریپو>
cd KSC
dotnet run --project samples/KSC.Excel.Sample.Web --urls http://localhost:5199 -- --environment
Development
```

مرورگر (Chrome/Edge) را باز کنید:

```
http://localhost:5199/excel-designer
```

> بدون `--environment Development` طراح لاگین می‌خواهد و در حالت ناشناس خطای ۴۰۱ می‌دهد؛ این رفتار امنیتی عمدی است (بخش ۱۵).

صفحه‌ی اصلی نمونه (`http://localhost:5199/`) چند لینک دارد: طراح، نمایش در صفحه (`/Home/Viewer`)، دانلود Excel (`/Home/Download`) و PDF (`/Home/Pdf`).

ب) داخل برنامه‌ی ASP.NET Core خودتان

```
builder.Services.AddKscExcel();
var app = builder.Build();
app.MapKscExcelDesigner("/excel-designer").RequireAuthorization();   // حتماً پشت لاگین
```

سپس `https://your-app/excel-designer` .

ج) داخل برنامه‌ی .NET Framework (MVC 5)

```
// Global.asax.cs
KscExcel.Initialize(o => o.TemplateDirectory = "ExcelTemplates");
// RouteConfig.cs
routes.MapKscExcelDesigner();      // /excel-designer
```

د) برنامه‌ی ویندوزی (بدون مرورگر و وب‌سایت)

```
dotnet publish src/KSC.Excel.Designer.Windows -c Release -r win-x64 --self-contained false -o
artifacts/KSC.Excel.Designer
artifacts\KSC.Excel.Designer\KSC.Excel.Designer.exe
```

جزئیات: بخش ۹.

---

۳) نصب و پیش‌نیازها

مورد	نیاز
.NET	SDK 8 یا 10 (برای ساخت)؛ برای اجرا Runtime همان نسخه
.NET Framework	4.7.2 یا 4.8 برای آداپتر MVC (هسته از 4.6.1)
مرورگر طراح	Chrome/Edge/Firefox جدید
Excel	**لازم نیست**. فایل‌ها در Excel 2016+ دسکتاپ بررسی شده‌اند؛ LibreOffice/Excel Online/Google Sheets آزمایش نشده‌اند
برنامه‌ی ویندوزی	ویندوز ۱۰/۱۱، .NET 10 Desktop Runtime، WebView2 Runtime (روی ویندوز ۱۱ هست)
PDF روی لینوکس	فونت فارسی (مثل Vazirmatn) و بسته‌ی `SkiaSharp.NativeAssets.Linux.NoDependencies`

افزودن به پروژه

تا زمان انتشار روی NuGet، پروژه‌ها را مستقیم ارجاع دهید:

```
<ItemGroup>
  <ProjectReference Include="..\KSC.Excel.AspNetCore\KSC.Excel.AspNetCore.csproj" />
  <ProjectReference Include="..\KSC.Excel.Pdf\KSC.Excel.Pdf.csproj" />   <!-- اختیاری: PDF -->
</ItemGroup>
```

پس از انتشار روی NuGet: `dotnet add package KSC.Excel.AspNetCore` (و `KSC.Excel.Pdf`).

---

۴) تور طراح

۴-۱) اجزای صفحه

بخش	محتوا
نوار بالا	نام قالب · **جدید** · **باز کردن** · بازگردانی/جلو · **ذخیره** · **پیش‌نمایش** · **دریافت xlsx** · ورود از xlsx · **PDF** (وقتی `UsePdf()` فعال باشد) · خروجی/ورود JSON
ریبون	فونت و اندازه · ضخیم/مورب/زیرخط/خط‌خورده · رنگ متن و پس‌زمینه · کادر · تراز افقی/عمودی · شکستن متن · ادغام · تورفتگی · قالب عدد · ساختار (درج/حذف سطر و ستون) · فریز · پاک‌کردن قالب · **افزودن نوار تکرار** · تصویر · نمودار · قالب شرطی · اعتبارسنجی
نوار فرمول	آدرس سلول · `fx` · مقدار/فرمول

بخش	محتوا
پنل چپ (در RTL سمت راست)	**فیلدها** · **مجموعه و پارامتر** · **داده‌ی نمونه**
شبکه	سلول‌ها شبیه Excel؛ نوارها با برچسب رنگی کنار شماره‌ی سطر
پنل راست	**سلول** · **برگه** · **نوارها** · **قوانین** · **چاپ** · **تصاویر و نمودار**
پایین	تب برگه‌ها · وضعیت انتخاب

۴-۲) میان‌برها

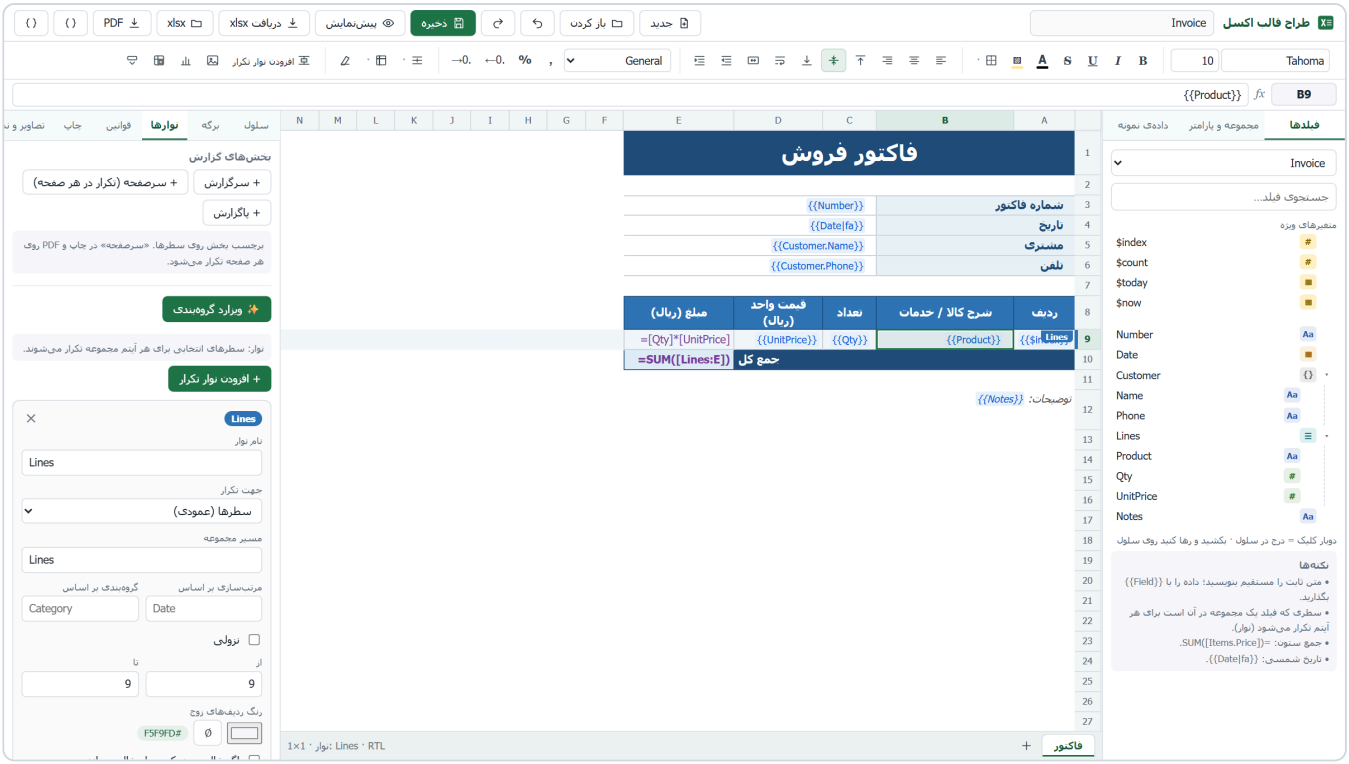
کلید	عمل	کلید	عمل
`Ctrl+S`	ذخیره	`F2` / `Enter`	ویرایش سلول
`Ctrl+Z` / `Ctrl+Y`	بازگردانی / جلو	`Alt+Enter`	سطر جدید داخل سلول
`Ctrl+B / I / U`	ضخیم / مورب / زیرخط	`Tab` / `Shift+Tab`	سلول بعدی / قبلی
`Ctrl+C / X / V`	کپی / برش / چسباندن (از/به Excel هم)	`Delete`	پاک کردن محتوا
`Ctrl+A`	انتخاب همه	`Ctrl+فلش`	پرش به لبه‌ی داده
`Shift+فلش/کلیک`	گسترش انتخاب	کلیک راست	منوی سلول/سطر/ستون

در برگه‌ی راست‌به‌چپ، فلش راست = ستون قبلی (مثل Excel). عرض ستون و ارتفاع سطر با کشیدن لبه‌ی سرستون/سرسطر تغییر می‌کند.

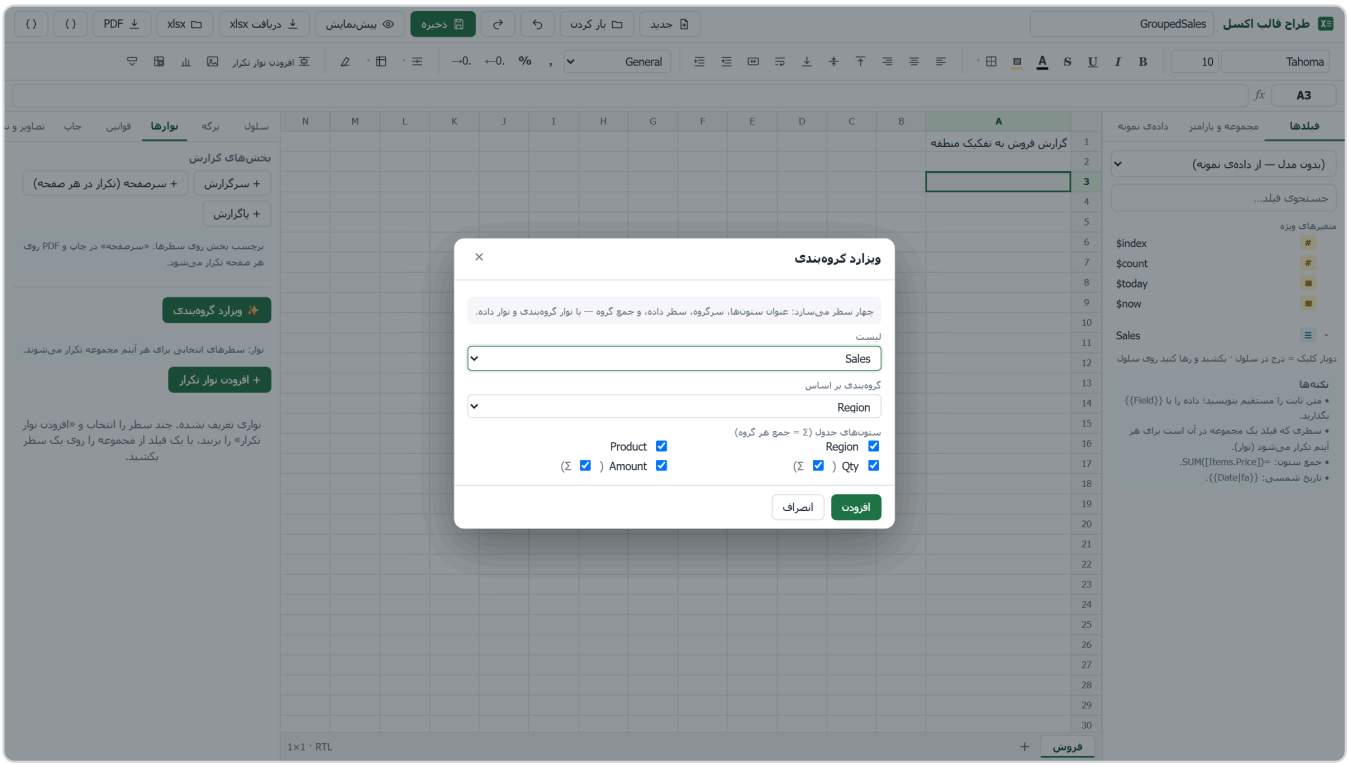
۴-۳) پنل فیلدها و داده‌ی نمونه

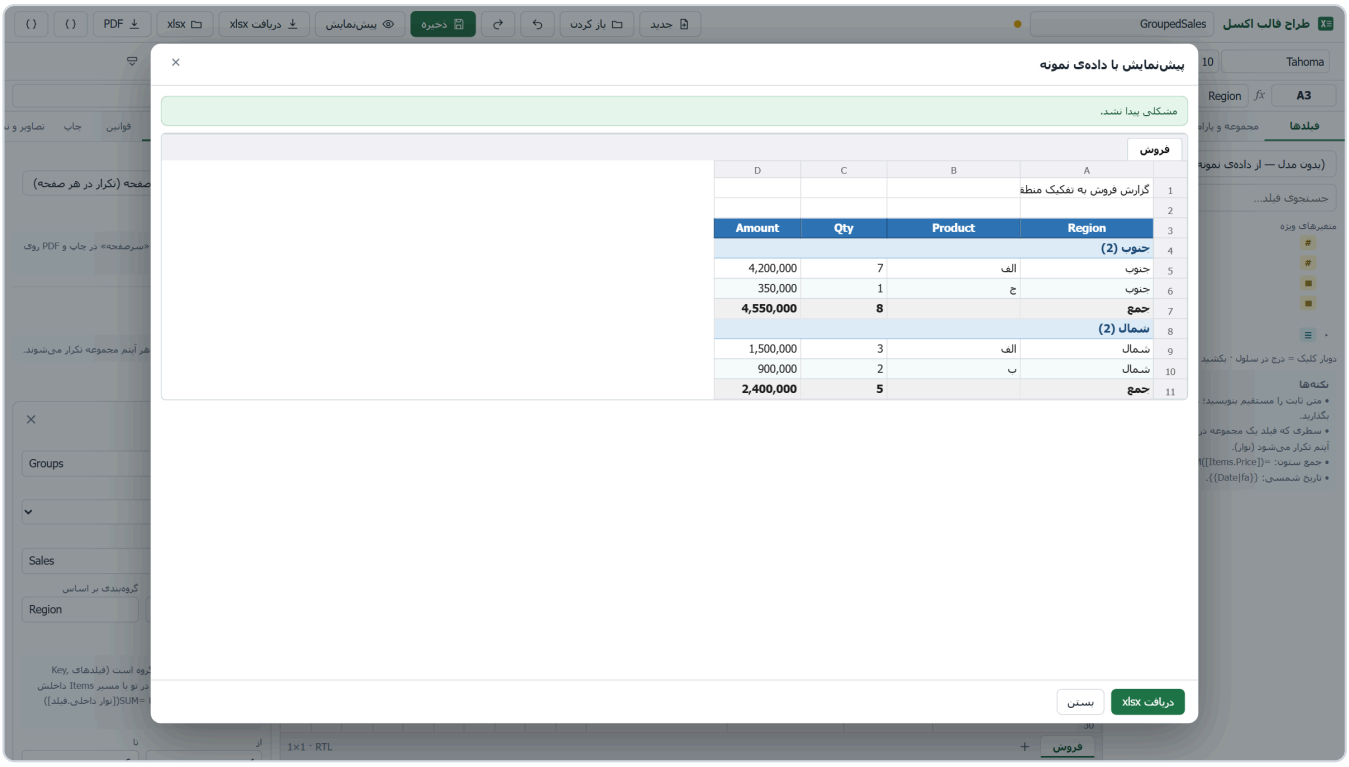
- فیلدها از **مدل ثبت‌شده** (`AddModel<T>`) یا از **داده‌ی نمونه‌ی JSON** استنتاج می‌شوند.
- فیلد را **روی سلول بکشید** یا دوبار کلیک کنید. فیلدی که عضو یک لیست است، **نوار تکرار همان سطر را خودکار می‌سازد**.
- روی گره‌ها: دکمه‌ی **Σ** (جمع ستون عددی) و **ش** (تاریخ شمسی).
- تب «داده‌ی نمونه»: JSON نمونه‌ی داده‌ای که به قالب می‌دهید (فقط برای پیش‌نمایش طراح).

۴-۴) نوارها و بخش‌ها



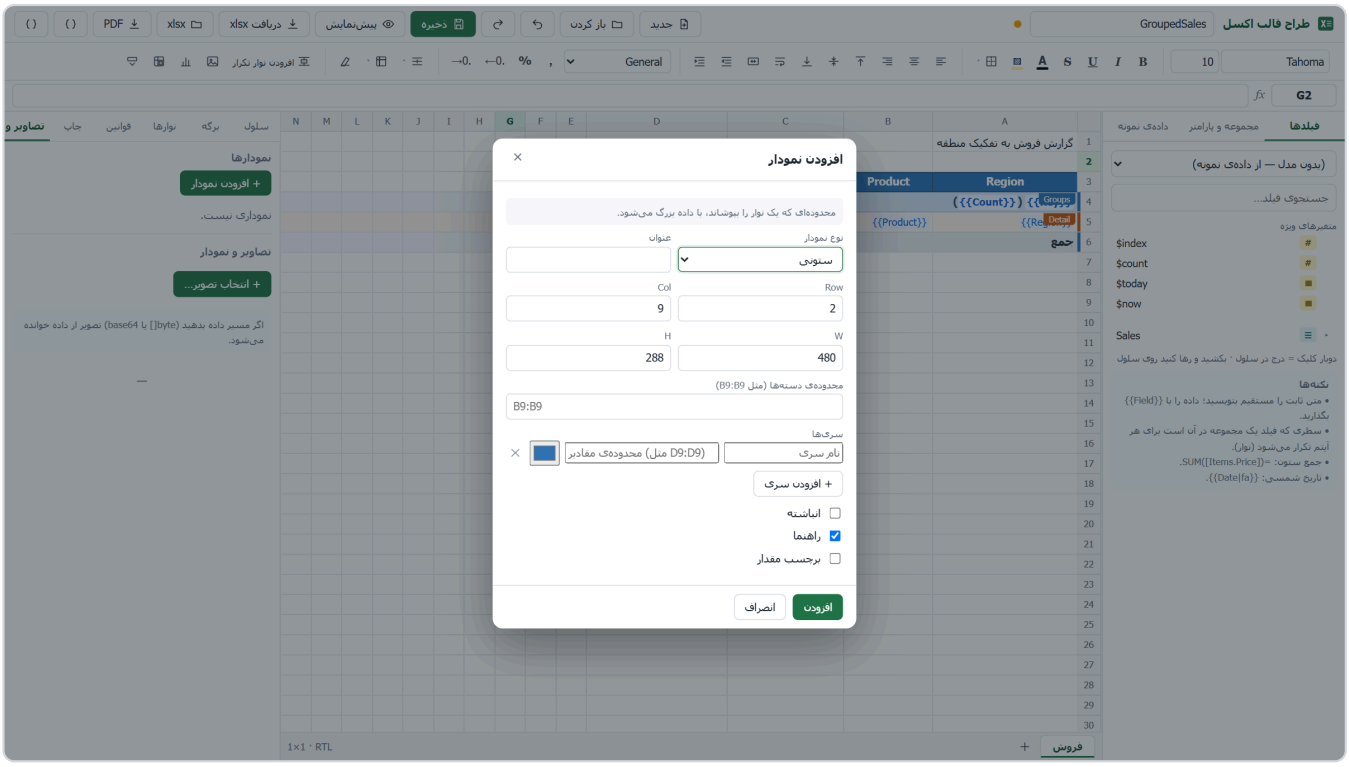
- **نوار تکرار:** مسیر مجموعه (`Lines`)، نام، سطرها، رنگ ردیف زوج، «اگر خالی بود»، **مرتب‌سازی**، **گروه‌بندی**، جهت (سطر/ستون).
- **بخش‌ها:** سرگزارش · **سرصفحه** (در چاپ و PDF روی هر صفحه تکرار می‌شود) · پاگزارش — برچسب رنگی روی سطرها.
- **ویزارد گروه‌بندی ✨:** لیست و فیلد گروه‌بندی و ستون‌ها را انتخاب کنید؛ چهار سطر (عنوان ستون‌ها، سرگروه، داده، جمع گروه) و نوارها خودکار ساخته می‌شود.





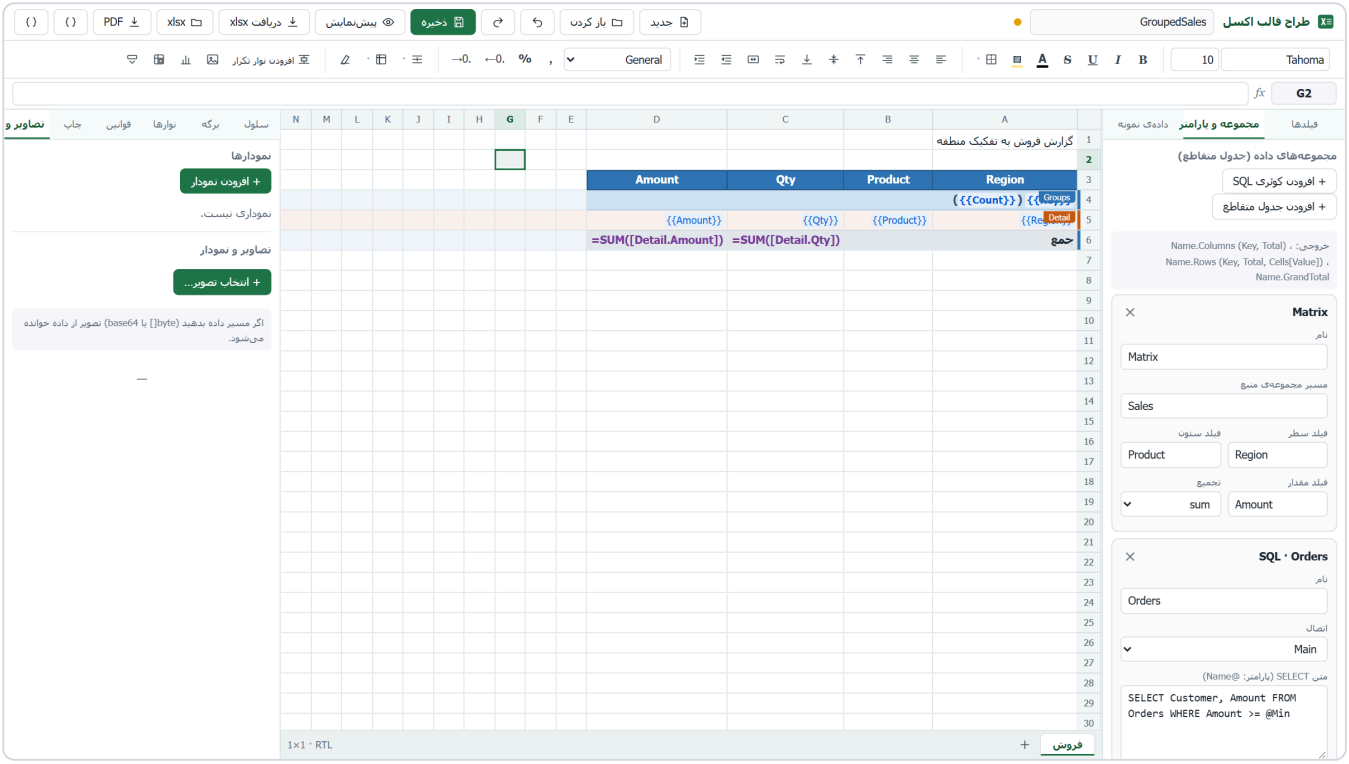
۴-۵) پنل‌های دیگر

- **سلول:** مقدار/فرمول، نوع مقدار (خودکار/همیشه متن)، پیوند، جهت متن.
- **برگه:** نام، راست‌به‌چپ، خطوط شبکه، رنگ تب، زوم، عرض/ارتفاع پیش‌فرض، «تکرار برگه برای هر آیتم»، فریز، فیلتر خودکار، محافظت.
- **قوانین:** قالب شرطی (مقدار سلول، فرمول، شامل متن، تکراری، Top/Bottom، طیف رنگ، نوار داده) و اعتبارسنجی (فهرست، عدد، تاریخ، طول، فرمول).
- **چاپ:** جهت، کاغذ، جا دادن عرض، حاشیه، سر/پاصفحه (`&P &N &D &T &A &F`)، تکرار سطرها، محدوده‌ی چاپ.
- **تصاویر و نمودار:** لوگوی ثابت یا از داده (`byte[]`/base64)؛ نمودار با نوع، عنوان، محدوده‌ی دسته‌ها و سری‌ها.

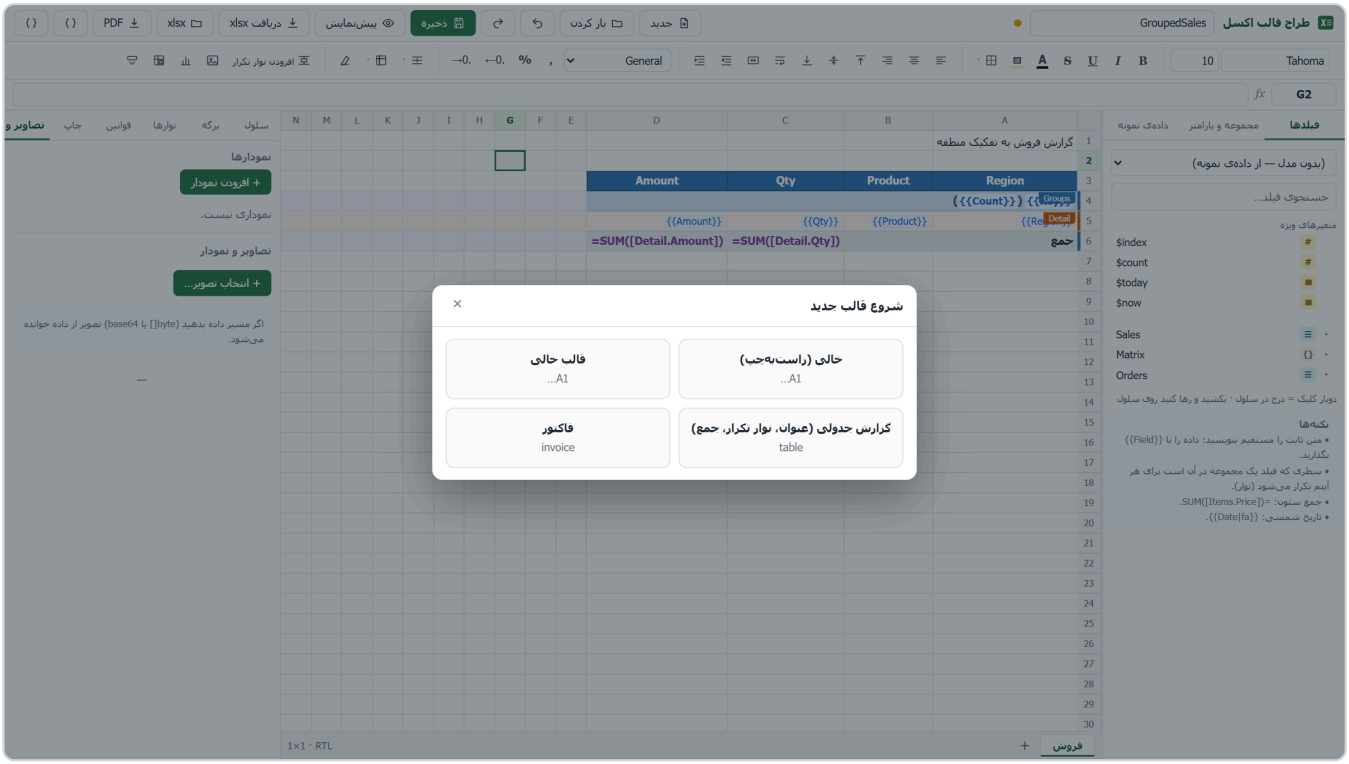


۴-۶) مجموعه و پارامتر

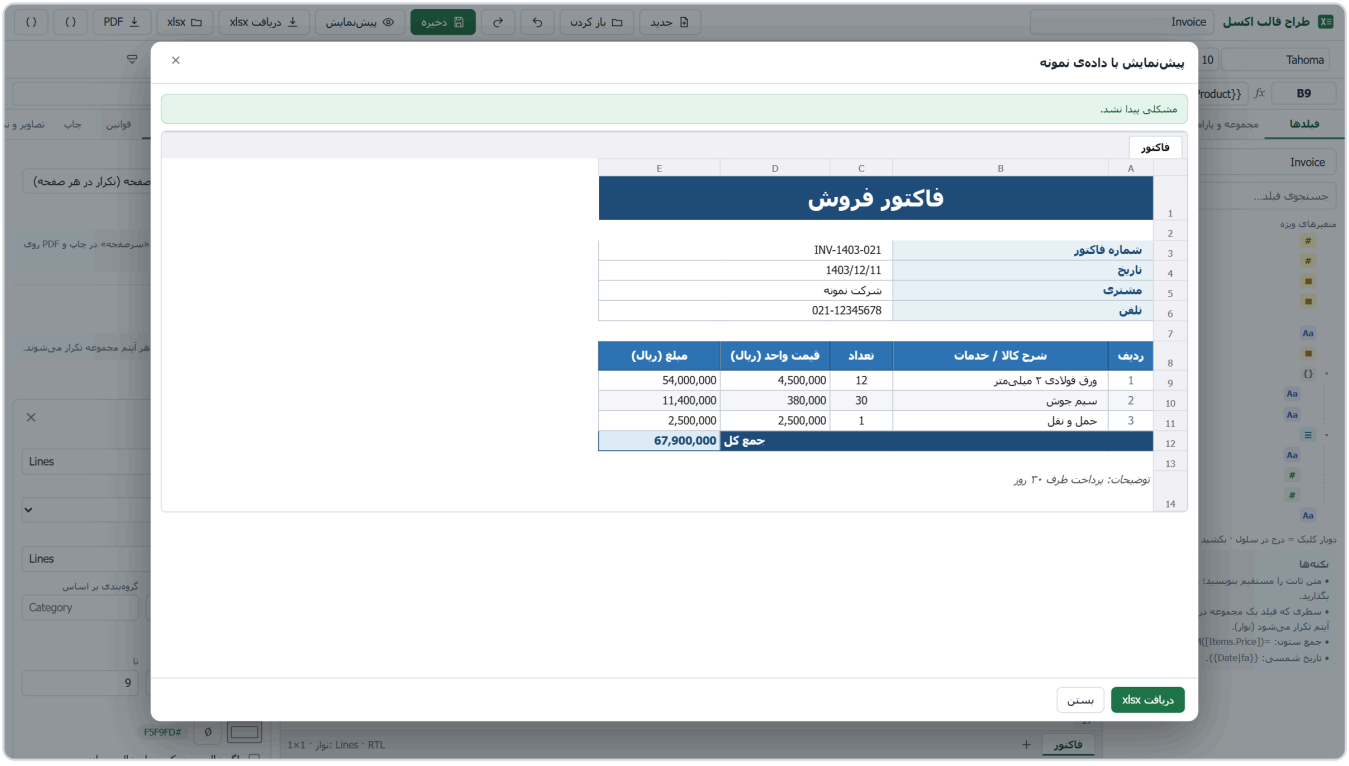
- **جدول متقاطع** (pivot)، **کوئری SQL**، **پارامتر**.



۴-۷) شروع قالب جدید و پیش‌نمایش



پیش‌نمایش قالب را با داده‌ی نمونه اجرا می‌کند، همه‌ی برگه‌ها را با تب نشان می‌دهد و **هشدارهای قالب** را فهرست می‌کند:



۴-۸) ذخیره‌ی خودکار و جابه‌جایی

پیش‌نویس هر تغییر در مرورگر نگه داشته می‌شود و پس از بستن اتفاقی صفحه «بازیابی؟» می‌پرسد. ذخیره‌ی واقعی با **ذخیره** روی سرور است. **خروجی/ورود JSON** برای جابه‌جایی بین محیط‌هاست. آدرس `?t=Invoice` قالب را

مستقیم باز می‌کند.

---

۵) آموزش گام‌به‌گام: ساخت یک فاکتور

هدف: فاکتور فارسی با مشخصات مشتری، جدول اقلام، جمع کل و خروجی Excel/PDF. (نتیجه همان قالب `Invoice` نمونه است.)

۱. مدل را ثبت کنید (یک‌بار در `Program.cs`):

```
builder.Services.AddKscExcel(o => { o.AddModel<Invoice>("Invoice"); });
public class Invoice {
    public string Number { get; set; } = ""; public DateTime Date { get; set; }
    public Customer Customer { get; set; } = new(); public List<InvoiceLine> Lines { get; set; } =
new();
    public string Notes { get; set; } = "";
}
public class Customer { public string Name { get; set; } = ""; public string Phone { get; set; } = "";
}
public class InvoiceLine { public string Product { get; set; } = ""; public int Qty { get; set; }
public decimal UnitPrice { get; set; } }
```

۲. طراح را باز کنید ← **جدید** ← «خالی (راست‌به‌چپ)». در پنل فیلدها مدل `Invoice` را انتخاب کنید.

۳. عنوان: سلول‌های A1 تا E1 را انتخاب کنید ← **ادغام** ← متن «فاکتور فروش» ← فونت ۲۰ ضخیم، رنگ متن سفید، پس‌زمینه‌ی آبی تیره، تراز وسط.

۴. مشخصات: `Customer.Name`، `Date`، `Number` را از پنل فیلدها روی سلول‌های ستون راست بکشید. برای تاریخ شمسی روی گره‌ی `Date` دکمه‌ی **ش** را بزنید (`{{Date|fa}}`). کنار هر کدام برچسب بنویسید («شماره فاکتور»، ...).

۵. جدول: ردیف سرستون را بنویسید (ردیف، شرح کالا، تعداد، قیمت واحد، مبلغ) و ظاهرش را بدهید (پس‌زمینه، کادر، ضخیم).

۶. ردیف داده: در سطر بعد `{{$index}}`، سپس `Lines.Product`، `Lines.Qty`، `Lines.UnitPrice` را بکشید. **نوار خودکار ساخته می‌شود** (`Lines` برچسب رنگی کنار سطر). ستون «مبلغ» را فرمول بنویسید: `=[Qty]*[UnitPrice]`.

۷. جمع کل: سطر زیر نوار: «جمع کل» و `=SUM([Lines:E])` (ستون E = مبلغ). قالب عدد `#,##0` بدهید.

۸. ظاهر: در تب نوارها «رنگ ردیف‌های زوج» را روشن کنید؛ برگه ← خطوط شبکه را خاموش کنید.

۹. پیش‌نمایش ← خطاها/هشدارها را ببینید ← **ذخیره** با نام `Invoice`.

۱۰. استفاده در کد:

```
public IActionResult Invoice(int id) => this.ExcelFile("Invoice", _repo.Get(id), $"invoice-
{id}.xlsx");
```

> قالب‌ها فایل `ExcelTemplates/Invoice.kxl.json` هستند؛ در Git نگه دارید.

۶) زبان قالب

۶-۱) متن، عدد، مقدار

- متن ثابت: همان‌طور که تایپ می‌کنید. عدد ثابت (`123`) عدد است؛ `0912…` و رشته‌ی بیش از ۱۵ رقم **متن** می‌ماند. (پنل سلول ← «همیشه متن»).
- داده: `{{مسیر}}` مثل `{{Lines[0].Product}}` ، `{{Customer.Name}}` ؛ بدون حساسیت به حروف.
- اگر کل سلول **فقط یک** `{{…}}` باشد، **نوع** حفظ می‌شود (عدد/تاریخ/منطقی)؛ ترکیب با متن رشته می‌شود.
- مقدار `null` = سلول خالی (`|default:` برای جایگزین).

۶-۲) متغیرهای ویژه

متغیر	معنی
`{{$index}}`	شماره‌ی ردیف (از ۱) داخل نوار
`{{$count}}`	تعداد آیتم‌های نوار (در Streaming ناشناخته)
`{{$now}}` / `{{$today}}`	امروز / اکنون (سرور)
`{{$item}}` / `{{$parent}}` / `{{$root}}`	داده‌ی ریشه / والد / آیتم جاری
`{{$p.Name}}`	پارامتر گزارش

داخل نوار، اگر نامی روی آیتم جاری نبود به والد و ریشه نگاه می‌شود.

۶-۳) فیلترها `{{مسیر | فیلتر:آرگومان}}`

فیلتر	کار	مثال
`fa`	تاریخ شمسی (متن)	`{{Date|fa}}` → `1404/01/01` ، `{{Date|fa:yyyy/MM/dd HH:mm}}`
`fadigits`	ارقام فارسی	`{{N|fadigits}}` → `۱۲۳۴`
`format:الگو`	قالب .NET	`{{Date|format:yyyy-MM-dd}}`
`round:n`	گرد کردن	`{{P|round:1}}`
`upper` `lower` `trim`	حروف/فاصله	
`default:متن`	جایگزین خالی	`{{X|default:-}}`
`yesno:بله/خیر`	منطقی به متن	`{{Active|yesno:بله/خیر}}`
`join:،`	اتصال لیست	`{{Tags|join:, }}`
`truncate:n`	کوتاه کردن	
فیلتر سفارشی	ثبت با `AddFilter`	`{{Rating|stars}}`

> تاریخ بدون فیلتر، **تاریخ میلادیِ واقعی** با قالب `yyyy/mm/dd` است (قابل مرتب‌سازی)؛ `|fa` متن شمسی می‌دهد.

۶-۴) تجمیع و عبارت محاسباتی

- تجمیع: `Max` ، `Min` ، `Avg` ، `{{Sum(Lines.Qty)}}` ، `{{Count(Lines)}}` (مقدار ثابت).
- عبارت: `{{Qty * UnitPrice}}` · `{{IF(Total > 1000, 'VIP', 'عادی')}}` · `{{ROUND(Price * 1.09, 0)}}` · `{{FA(Date, 'yyyy/MM/dd')}}` · `{{COALESCE(Nick, Name)}}` · `{{FirstName & ' ' & LastName}}` .
- عملگرها: `+ - * / % ^ & = <> < > <= >=` ؛ توابع: `IF IFERROR COALESCE ROUND ABS MIN MAX SUM AVG LEN UPPER LOWER TRIM CONCAT AND OR NOT TODAY NOW YEAR MONTH DAY FA` و توابع سفارشی.

۶-۵) نوارهای تکرار

سناریو	قالب
لیست ساده	سطر با `{{Product}}` … و نوار `Lines` روی همان سطر
چند سطر برای هر آیتم	نوار روی چند سطر
تو در تو	نوار `Groups` روی سطرهای ۱–۳ و نوار `Lines` روی سطر ۲ (مسیر نسبی `Lines`)
لیست خالی	سطرها حذف می‌شوند؛ «اگر خالی بود یک سطر بماند»
ردیف‌های زوج	رنگ زوج (zebra)، به‌جز سلول‌های با پس‌زمینه‌ی صریح
مرتب‌سازی/گروه‌بندی	`GroupBy` , `SortBy` روی نوار؛ هر گروه فیلدهای `Items` , `Count` , `Key` دارد

گروه‌بندی با جمع هر گروه:

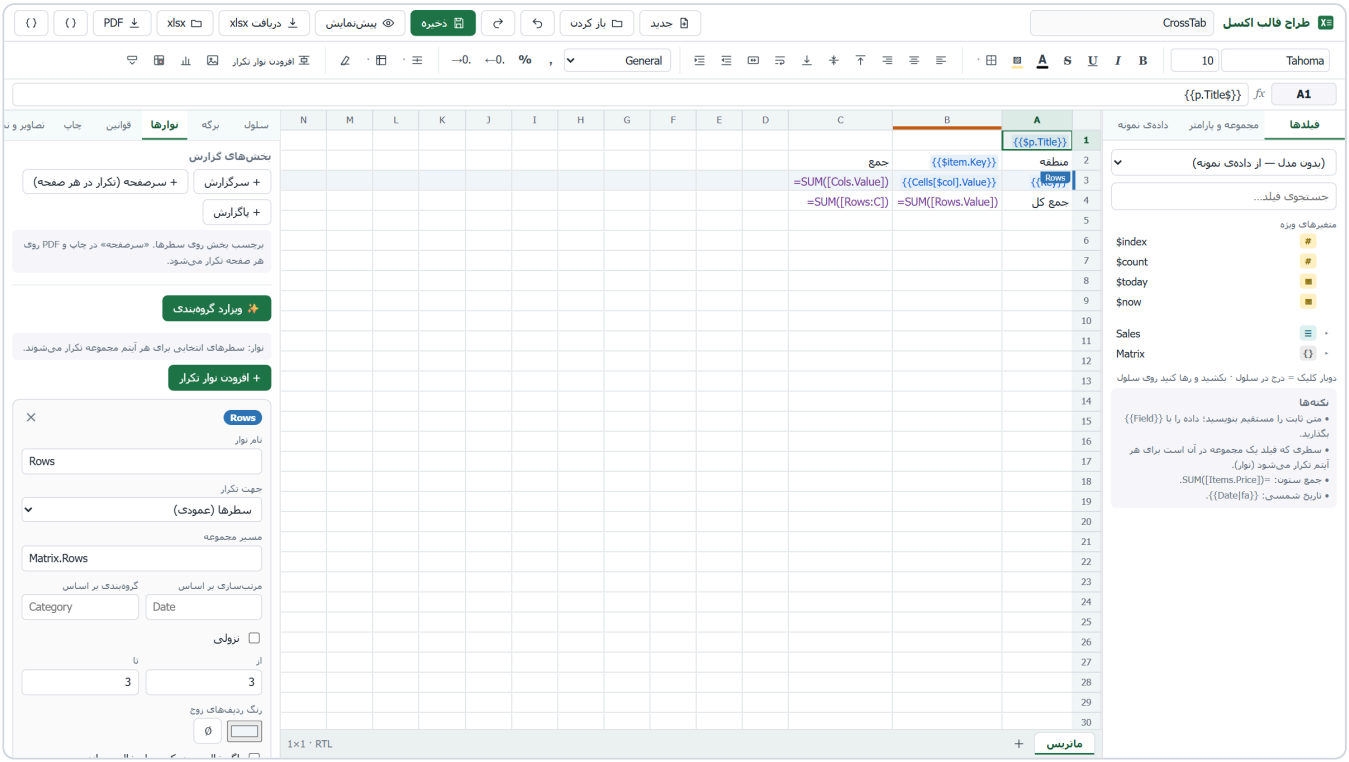
سطر	محتوا	نوار
۱	`{{Key}} ({{Count}})`	`Groups` (مسیر `Sales` ، گروه‌بندی `Region`) روی سطرهای ۱–۳
۲	`{{Product}}` · `{{Qty}}`	`Detail` (مسیر `Items`) روی سطر ۲
۳	`=SUM([Detail.Qty])` · جمع	(جزو `Groups`)

(ویزارد همین را خودکار می‌سازد.)

۶-۶) جدول متقاطع (Cross-tab)

1. **مجموعه و پارامتر** ← «افزودن جدول متقاطع»: نام `Matrix` ، منبع `Sales` ، سطر `Region` ، ستون `Product` ، مقدار `Amount` ، تجمیع `sum` .
2. نوار ردیفی `Rows` (مسیر `Matrix.Rows`) روی سطر داده.
3. نوار **ستونی** `Cols` (مسیر `Matrix.Columns` ، جهت «ستون‌ها»، ستون ۲ تا ۲).

سلول	محتوا
سرستون (داخل نوار ستونی)	`{{$item.Key}}`
داده (داخل نوار ستونی)	`{{Cells[$col].Value}}`
جمع هر ردیف	`=SUM([Cols.Value])`
جمع هر ستون	`=SUM([Rows.Value])` (داخل نوار ستونی)
جمع کل	`=SUM([Rows:C])`



در کد: `Pivot.Build(data, "Region", "Product", "Amt", "sum")`.

۶-۷) فرمول‌ها و توکن‌ها

هر سلول `=...` فرمول Excel است و زنده می‌ماند. توکن‌های ویژه (هنگام تولید به آدرس واقعی تبدیل می‌شوند):

توکن	معنی	مثال
`[Field]`	سلول همان فیلد در **همان سطر**	`=[Qty]*[Price]` → `=C9*D9`
`[Band.Field]`	کل ستون فیلد در نوار	`=SUM([Lines.Qty])` → `=SUM($C$9:$C$11)`
`[Band:E]`	کل ستون `E` در نوار	`=SUM([Lines:E])`
`[ColBand.Field]`	بازه‌ی **افقی** همان سطر در نوار ستونی	`=SUM([Cols.Value])`

`{{…}}` داخل فرمول هم جایگزین می‌شود: `=C9*{{TaxRate}}`. نوار خالی ← توکن `0`. **مقدار ذخیره‌شده** برای توابع رایج محاسبه می‌شود (`SUM AVERAGE MIN MAX COUNT COUNTA ABS ROUND IF IFERROR AND OR NOT CONCAT LEN`)

`UPPER LOWER TRIM SUMIF COUNTIF`)؛ بقیه‌ی توابع Excel (مثل `VLOOKUP`) نوشته می‌شوند و در Excel درست کار می‌کنند. ارجاع ثابت `A1` در فرمول با جابه‌جایی سطر/ستون **تنظیم نمی‌شود**؛ از توکن‌ها استفاده کنید.

۸-۶) تکرار برگه، لینک، تصویر پویا

- **تکرار برگه:** پنل برگه ← «تکرار برگه برای هر آیتم» = `Branches` (نام برگه می‌تواند `{{Name}}` داشته باشد).
- **لینک:** پنل سلول ← پیوند: `https://site/{{Id}}` یا `#Sheet2!A1`.
- **تصویر پویا:** مسیر داده‌ی تصویر (`Company.Logo` از نوع `byte[]` یا base64).

---

۷) راهنمای برنامه‌نویس — (8/10) .NET

۷-۱) راه‌اندازی

```
using KSC.Excel.AspNetCore;
using KSC.Excel.Pdf;

builder.Services.AddControllersWithViews();
builder.Services.AddKscExcel(o =>
{
    o.AddModel<Invoice>("Invoice");      // فیلدها و داده‌ی نمونه‌ی طراح
    o.UsePdf();                          // دکمه‌ی PDF (اختیاری) در طراح
});

var app = builder.Build();
app.UseRouting();
app.UseAuthentication(); app.UseAuthorization();

app.MapKscExcelDesigner("/excel-designer").RequireAuthorization("Admin");  // طراح
app.MapKscExcelDownloads();                                                 // لینک‌های Viewer
app.MapDefaultControllerRoute();
```

قالب‌ها: `{ContentRoot}/ExcelTemplates/*.kxl.json` (پیش‌فرض). برای انتشار: `<Content Include="ExcelTemplates\**" CopyToOutputDirectory="PreserveNewest" />`.

۷-۲) تولید فایل

```
public class ReportsController : Controller
{
    private readonly IExcelReportService _excel;
    public ReportsController(IExcelReportService excel) => _excel = excel;

    public IActionResult Invoice(int id) =>
        this.ExcelFile("Invoice", _repo.Get(id), $"invoice-{id}.xlsx");              // دانلود

    public IActionResult Sales(DateTime from) =>
        this.ExcelFile("Sales", _repo.Query(from), new Dictionary<string, object> { ["From"] = from },
"sales.xlsx");   // با پارامتر

    public IActionResult Big() => this.ExcelFileStreaming("Big", _repo.StreamRows(), "big.xlsx");   //
Streaming

    public IActionResult InvoicePdf(int id) =>
        File(_excel.RenderPdf("Invoice", _repo.Get(id)), "application/pdf", "invoice.pdf");        //
PDF
}
```

Minimal API: `app.MapGet("/r", (IExcelReportService x) => x.File("Invoice", data, "invoice.xlsx"));`

بدون وب (Job/سرویس):

```
byte[] bytes = svc.Render("Invoice", model);
File.WriteAllBytes("invoice.xlsx", bytes);
svc.Render(svc.GetTemplate("Invoice"), model, outputStream);     // مستقیم روی Stream
```

۷-۳) نمایش در صفحه (Viewer)

```
@await Component.InvokeAsync("KscExcelViewer", new { template = "Invoice", data = Model, downloadName
= "invoice" })
@* یا: @addTagHelper *, KSC.Excel.AspNetCore  →  <vc:ksc-excel-viewer template="Invoice" data="Model"
/> *@
```

پارامتر	پیش‌فرض	توضیح
`template`	—	نام قالب
`data`	—	داده
`downloadName`	نام قالب	نام فایل
`showDownload`	`true`	دکمه‌ی دانلود
`showHeaders`	`false`	سرستون A,B,C و شماره‌ی سطر

پارامتر	پیش‌فرض	توضیح
`downloadLabel`	«دریافت فایل اکسل»	متن دکمه

لینک دانلود: توکن تصادفی ۱۴۴ بیتی، ۱۵ دقیقه اعتبار، متصل به کاربر لاگین، در حافظه‌ی همان سرور (حداکثر ۶۴ فایل).

۷-۴) داده‌ی ورودی

`data` می‌تواند باشد: کلاس/DTO، `JObject/JToken` ، `ExpandoObject` ، `Dictionary<string, object>` ، `DataTable/DataRow/DataView` ، یا ترکیبی. نام‌ها بدون حساسیت به حروف. برای Streaming داده را **تنبل** (`IEnumerable` با `DataReader` ، `yield`) بدهید.

۷-۵) `IExcelReportService`

متد	کار
`Render(name, data)` / `(name, data, parameters)`	xlsx `byte[]` از قالب ذخیره‌شده
`Render(template, data)` / `(template, data, Stream)`	از شیء قالب / روی Stream
`RenderStreaming(name|template, data, Stream, parameters?)`	Streaming؛ `bool` (false = fallback به رندر عادی)
`RenderHtml(name|template, data, showHeaders)`	HTML برای صفحه
`Bind(template, data, parameters?)`	`RenderedWorkbook` + هشدارها (`Warnings`)
`BindPreview(template, data, parameters?)`	مثل `Bind` ولی SQL را اگر مجاز نباشد رد می‌کند (طراح)
`Store` / `GetTemplate(name)`	قالب / ذخیره‌ساز
`svc.RenderPdf(...)` (بسته‌ی Pdf)	PDF

۷-۶) کد به‌جای طراح (Code-first)

```
var b = new TemplateBuilder("Report", "گزارش");
var head = b.Style("h", new StyleDef { Bold = true, Fill = "#2E75B6", Color = "#FFFFFF", H = "center"
});
b.Sheet("گزارش", s =>
{
    s.Cell("A1", "نام", head).Cell("B1", "مبلغ", head);
    s.Cell("A2", "{{Name}}").Cell("B2", "{{Amount}}");
    s.Band("Rows", "Rows", 2, 2, "#F5F9FD");
    s.Cell("A3", "جمع").Cell("B3", "=SUM([Rows.Amount])");
    s.Freeze(1);
}, rtl: true);
byte[] xlsx = svc.Render(b.Build(), new { Rows = data });
```

مناسب برای قالب‌های پیش‌فرض همراه برنامه (`store.Save(template)` در شروع).

۷-۷) ذخیره‌ساز قالب (`ITemplateStore`)

```
public sealed class DbTemplateStore : ITemplateStore
{
    public IList<TemplateInfo> List() => /* SELECT Name, Title, Modified */;
    public ExcelTemplate Get(string name) { var json = /* SELECT Json */; return json == null ? null :
ExcelTemplate.FromJson(json); }
    public void Save(ExcelTemplate t) { /* UPSERT t.Name, t.ToJson() */ }
    public bool Delete(string name) => /* DELETE */;
}
builder.Services.AddKscExcel(o => o.Store = new DbTemplateStore(...));
```

نام قالب را با `TemplateNames.IsValid(name)` اعتبارسنجی کنید.

۷-۸) تست خودکار قالب‌ها

```
var wb = svc.Bind(svc.GetTemplate("Invoice"), sampleData);
Assert.Empty(wb.Warnings);                                  // قالب بی‌هشدار
var bytes = XlsxWriter.ToBytes(wb);                          // بررسی ZipArchive و در صورت نیاز با
کنید
```

---

۸) راهنمای برنامه‌نویس — .NET Framework (MVC 5)

ارجاع به `KSC.Excel` و `KSC.Excel.Mvc` (آداپتر: `net472` / `net48`؛ هسته: از 4.6.1).

```
// Global.asax.cs
protected void Application_Start()
{
    AreaRegistration.RegisterAllAreas();
    KscExcel.Initialize(o => { o.TemplateDirectory = "ExcelTemplates"; o.AddModel<InvoiceVm>
("Invoice"); });
    KscExcel.Authorize = ctx => ctx.User.IsInRole("Admin");      // اختیاری؛ کاربر باید لاگین باشد
    RouteConfig.RegisterRoutes(RouteTable.Routes);
}

// RouteConfig.cs  (قبل از مسیر پیش‌فرض)
routes.MapKscExcelDesigner();          // /excel-designer
routes.MapKscExcelDownloads();         // برای Html.KscExcelViewer (بعد از Initialize)
```

```
// Controller
using KSC.Excel.Mvc;
public ActionResult Invoice(int id) => this.Excel("Invoice", _svc.Get(id), "invoice.xlsx");
public ActionResult Sales() => this.Excel("Sales", data, new Dictionary<string, object> { ["From"] =
DateTime.Today }, "sales.xlsx");
public ActionResult Big() => this.ExcelStreaming("Big", reader, "big.xlsx");
```

```
@using KSC.Excel.Mvc
@Html.KscExcelViewer("Invoice", Model, "invoice")
```

- سرویس بدون DI: `KscExcel.Service` (همان `IExcelReportService`).
- IIS: Integrated Pipeline؛ اگر مسیر `/excel-designer/assets/*.js` ۴۰۴ داد `<modules runAllManagedModulesForAllRequests="true" />` یا `UrlRoutingModule` را بررسی کنید.
- **روی IIS Express واقعی راستی‌آزمایی شد** (System.Web + MVC 5.2.9 ،4.8 .NET Framework): نمونه‌ی `samples/KSC.Excel.Sample.Mvc5` طراح، نمایشگر، نمایشگر PDF و دانلود Excel/PDF را سرو می‌کند. اجرا: `iisexpress.exe /path:samples\KSC.Excel.Sample.Mvc5 /port:5290`.
- **PDF روی .NET Framework. کار می‌کند** (بسته‌ی `KSC.Excel.Pdf` از نوع `netstandard2.0` است): در `KscExcel.Initialize` بنویسید `o.UsePdf()` و بعد `this.Pdf("Invoice", model, "invoice.pdf")` یا `Html.KscPdfViewer(...)`. SkiaSharp در `web.config` به Binding Redirect نیاز دارد (System.Buffers، System.Memory، System.Runtime.CompilerServices.Unsafe 6.0.3.0، System.Numerics.Vectors) — نمونه آن‌ها را دارد.

---

۹) برنامه‌ی ویندوزی طراح

همان طراح وب داخل `WebView2` (**.NET 10 یا .NET Framework 4.8** — بسته به ماشین) و **API آن داخل همان پروسه** (`https://kxl.local`؛ بدون Kestrel و بدون پورت). ظاهر و **فرمت قالب دقیقاً یکی** است.

```
dotnet publish src/KSC.Excel.Designer.Windows -c Release -r win-x64 --self-contained false -o
artifacts/KSC.Excel.Designer
artifacts\KSC.Excel.Designer\KSC.Excel.Designer.exe           # یا با فایل:  KSC.Excel.Designer.exe
"C:\path\Invoice.kxl.json"
```

- قالب‌ها پیش‌فرض در `Documents\KSC Excel Templates`؛ منوی **فایل ← انتخاب پوشه‌ی قالب‌ها** پوشه‌ی پروژه‌ی خودتان (مثلاً `ExcelTemplates` داخل Git) را می‌دهد. تنظیم در `%APPDATA%\KSC.Excel.Designer\settings.json`.
- منوها: قالب جدید · **باز کردن فایل** `.kxl.json` · ذخیره · ذخیره به‌عنوان JSON · **ورود از فایل Excel** · **خروجی Excel و PDF با داده‌ی نمونه** · انتخاب/نمایش پوشه · بارگذاری مجدد · DevTools.
- محدودیت: `AddModel<T>` و اتصال SQL در دسکتاپ ثبت نمی‌شوند؛ داده‌ی نمونه‌ی JSON بدهید.
- بررسی خودکار: `KSC.Excel.Designer.exe --selftest out.txt`.

---

۱۰) خروجی PDF

```
using KSC.Excel.Pdf;
byte[] pdf = svc.RenderPdf("Invoice", model);                          // با پارامتر:
svc.RenderPdf("Sales", data, parameters)
ExcelPdf.Render(svc.Bind(template, data, null), stream, new PdfOptions { FontFile =
"/fonts/Vazirmatn.ttf" });
byte[] png = ExcelPdf.RenderPng(wb, pageNumber: 1);                    // پیش‌نمایش تصویری
```

فاکتور فروش

شماره فاکتور	INV-1403-021
تاریخ	1403/12/11
مشتری	شرکت نمونه
تلفن	021-12345678

ردیف	شرح کالا / خدمات	تعداد	قیمت واحد (ریال)	مبلغ (ریال)
1	ورق فولادی ۲ میلی‌متر	12	4,500,000	54,000,000
2	سیم جوش	30	380,000	11,400,000
3	حمل و نقل	1	2,500,000	2,500,000
			جمع کل	67,900,000

توضیحات: پرداخت ظرف ۳۰ روز

	پشتیبانی
متن فارسی/عربی	شکل‌دهی درست (HarfBuzz)، ترتیب مختلط فارسی+عدد+لاتین (Bidi)، برگه‌ی RTL
سلول	پس‌زمینه، کادر، فونت/اندازه/ضخیم/مورب/زیرخط، تراز، شکستن متن، تورفتگی، ادغام، قالب عدد، فرمول
صفحه	کاغذ A4/A3/A5/Letter/Legal، جهت، حاشیه، مقیاس/جا دادن عرض، **تکرار سطرهای عنوان و سرصفحه**، سر/پاصفحه (`&P &N &D &T &A &F &L &C &R`)، صفحه‌بندی
تصویر/نمودار	تصویر؛ نمودار ستونی، میله‌ای، خطی، ناحیه‌ای، دایره‌ای
لینک	`http/https/mailto` قابل کلیک

ناپشتیبانی: پیکان‌های فیلتر خودکار، کامنت سلول. ظاهر بسیار نزدیک است ولی پیکسل‌به‌پیکسل مثل «ذخیره به‌عنوان Excel «PDF نیست.

فونت: فونت سلول ← Segoe UI/Arial/Noto/DejaVu ← (Tahoma) `FallbackFontFamily` ← فونت‌منیجر سیستم. روی **لینوکس/Docker** فونت فارسی نصب کنید یا `PdfOptions.FontFile` بدهید و `SkiaSharp.NativeAssets.Linux.NoDependencies` را اضافه کنید. سقف صفحه: (۲۰۰۰) `PdfOptions.MaxPages`.

۱۰-۱) نمایشگر PDF داخل صفحه

گزارش را به‌صورت PDF داخل صفحه نشان بدهید (نمایشگر PDF خود مرورگر: صفحه‌به‌صفحه، بزرگ‌نمایی، جستجو، چاپ) همراه دکمه‌های **دریافت PDF / دریافت Excel / چاپ**. بدون کتابخانه‌ی جاوااسکریپت و بدون CDN.

```
@* ASP.NET Core *@
@await Component.InvokeAsync("KscPdfViewer", new { template = "Invoice", data = Model, downloadName =
"invoice", showExcel = true, height = "75vh" })

@* .NET Framework MVC 5 *@
@Html.KscPdfViewer("Invoice", Model, "invoice", height: "75vh", showExcel: true)
```

پیش‌نیاز: `options.UsePdf()` (بسته‌ی KSC.Excel.Pdf) و مسیر دانلود (/ `app.MapKscExcelDownloads()` `routes.MapKscExcelDownloads()`). یک‌خطی در کنترلر:

```
return this.PdfFile("Invoice", model, "invoice.pdf");                    // ASP.NET Core  (inline:
true = نمایش در مرورگر)
return this.Pdf("Invoice", model, "invoice.pdf");                        // MVC 5         (inline:
true = نمایش در مرورگر)
app.MapGet("/r", (IExcelReportService s, KscExcelOptions o) => s.Pdf(o, "Invoice", model));   //
Minimal API
```

پارامترهای نمایشگر: `parameters`، `data`، `template` (پارامترهای گزارش)، `height` (CSS)، `downloadName`، `showExcel`، `showToolbar`.

۱۰-۲) آنچه PDF پشتیبانی می‌کند

همه‌ی جدول بالا **به‌علاوه‌ی**: قالب شرطی (همان موتور پیش‌نمایش HTML)، متن چرخیده و عمودی، سرریز متن به سلول‌های خالی کناری (مثل Excel)، کوچک‌شدن متن تا جا شود، نوار داده / طیف رنگ، **بارکد و QR به‌صورت شکل برداری تیز**، **واترمارک**، **شکست صفحه برای هر گروه**.

۱۰-۳) بارکد و QR Code

سلول را انتخاب کنید، در پنل «سلول» گزینه‌ی **بارکد** را بزنید (`QR Code` ، `QR Code (L)` ، `QR Code (H)` ، `Code 128`) و محتوا را معمول بنویسید: `{{Number}}` یا `https://pay.example.com/{{Id}}`. متن کدگذاری و به‌صورت تصویری که کل سلول را پر می‌کند رسم می‌شود (برای بارکد بزرگ‌تر سلول‌ها را ادغام کنید). داخل نوار تکرار (برای هر سطر یک کد)، در `.xlsx` (تصویر PNG)، در پیش‌نمایش HTML به‌صورت نشانگر و در **PDF به‌صورت شکل برداری** کار می‌کند.

- **QR:** حالت بایتی (UTF-8؛ پس فارسی هم می‌شود)، نسخه‌ی ۱ تا ۱۰ (حدود ۲۷۰ بایت در سطح L)، سطح تصحیح خطا `qr:L` ، `qr:M` ، `qr:Q` ، `qr:H` (L/M/Q/H؛ پیش‌فرض M). در تست‌ها با یک رمزگشای مستقل راستی‌آزمایی شده است.
- **Code 128:** نویسه‌های قابل چاپ ASCII؛ مقدار فقط‌رقمی با طول زوج خودکار از مجموعه‌ی فشرده‌ی C استفاده می‌کند.
- متنی که قابل کدگذاری نباشد هشدار می‌دهد و تصویری نمی‌سازد (بقیه‌ی گزارش ساخته می‌شود).
- قالب JSON: `{ "r": 14, "c": 1, "v": "{{Number}}", "bc": "qr" }`. سلول بارکد با نویسنده‌ی Streaming سازگار نیست.

۱۰-۴) واترمارک و شکست صفحه

- **واترمارک** (PDF): پنل چاپ ← *واترمارک*، یا `PdfOptions.Watermark = "DRAFT"` (بر قالب غلبه می‌کند). مورب، خاکستری روشن، روی هر صفحه، با هر خط.
- **شکست صفحه برای هر آیتم:** پنل نوارها ← *هر آیتم در صفحه‌ی جدید* (`BandDef.PageBreak`) — یک فاکتور / یک گروه در هر صفحه. در `.xlsx` به‌صورت شکست دستی صفحه (`rowBreaks`) نوشته و در صفحه‌بندی PDF اعمال می‌شود.

---

۱۱) گزارش‌های بسیار بزرگ (Streaming)

```
bool streamed = svc.RenderStreaming("BigReport", new { Title = "…", Items = repo.StreamRows() },
outputStream);
```

- اندازه‌گیری: **۱٬۰۰۰٬۰۰۰ سطر × ۶ ستون ≈ ۱۲٫۶ ثانیه، ≈ ۴۷ مگابایت حافظه‌ی اوج** (فایل ۳۰ مگابایت)؛ ۱۰۰ هزار سطر ≈ ۲ ثانیه. فقط `MaxRows` اعمال می‌شود.
- اگر قالب «قابل Streaming» نباشد، `RenderStreaming` به رندر عادی برمی‌گردد و `false` می‌دهد؛ دلیل‌ها: `StreamAnalyzer.Check(template)`. شروط: جمع هر نوار فقط **زیر** آن باشد، `[Field]` به سطر بعدی ارجاع ندهد، `{{$count}}` نباشد، سطرهای فریز بالای همه‌ی نوارها.
- تفاوت: رشته‌ها `inlineStr` و فرمول‌ها **بدون مقدار ذخیره‌شده** (Excel هنگام باز شدن حساب می‌کند).

- خطا وسط نوشتن، فایل ناقص در `output` می‌گذارد؛ آداپترها در فایل موقت می‌نویسند.

---

۱۲) منبع داده‌ی SQL، پارامتر و توابع سفارشی

پارامتر

تب **مجموعه و پارامتر** ← نام، نوع (`text/number/date/bool`)، پیش‌فرض؛ در قالب `{{$p.Title}}` ؛ در کد:

```
svc.Render("Sales", data, new Dictionary<string, object> { ["Title"] = "فروش بهار", ["From"] =
DateTime.Today });
```

SQL

```
o.AllowSql = true;                                                    // پیش‌فرض false
o.AddConnection("Sales", SqlClientFactory.Instance, connectionString);      // هر DbProviderFactory
o.SqlCommandTimeoutSeconds = 30; o.SqlMaxRows = 100000;
```

در طراح: «افزودن کوئری SQL» ← نام (`Orders`)، اتصال، `SELECT … WHERE Amount >= @Min` (پارامتر `@Min` از پارامتر هم‌نام بسته می‌شود؛ **هیچ‌وقت الحاق رشته**). «آزمایش کوئری» ستون‌ها را پیدا می‌کند. سپس نوار با مسیر `Orders`. **ایمنی:** فقط یک `SELECT` / `WITH` ؛ `INSERT/UPDATE/DELETE/DROP/EXEC/INTO/;` رد می‌شوند. این فقط لایه‌ی دوم است — **از لاگین فقط‌خواندنی استفاده کنید.** با `AllowSql=false` و قالب SQLدار، تولید با خطا متوقف می‌شود؛ پیش‌نمایش طراح از داده‌ی نمونه‌ی هم‌نام استفاده می‌کند.

REST / JSON / ارائه‌دهنده‌ی کد (هر منبع داده)

SQL یکی از منابع است؛ موتور از سه نوع مجموعه‌ی داده‌ی دیگر هم می‌تواند **هر چیزی** را بخواند. قالب فقط یک منبع را *نام* می‌برد؛ **اینکه منبع چیست در کد شما تعیین می‌شود** (طراح قالب نمی‌تواند موتور را به سرور دیگری وصل کند).

```
builder.Services.AddKscExcel(o =>
{
    // ۱) کد خودتان داده را از پارامترهای گزارش برمی‌گرداند (EF، Dapper، فراخوانی سرویس...)
    o.AddProvider("Orders", p => db.Orders.Where(x => x.Year == (int)p["Year"]).ToList());

    // ۲) نقطه‌ی REST: آدرس سرور و هدرها (کلید API) ثابت؛ قالب فقط مسیر را می‌دهد
    o.AddHttpEndpoint("Crm", "https://crm.example.com/api/", new Dictionary<string, string> { ["X-Api-
Key"] = key }, timeoutSeconds: 30);
});
```

در طراح (تب «مجموعه و پارامتر»): **+ افزودن داده‌ی JSON / REST** یا **+ افزودن ارائه‌دهنده‌ی کد.**

نوع مجموعه‌ی داده	فیلدها	مثال
`provider`	`connection` = نام ارائه‌دهنده	`Orders`
`json` + نقطه‌ی REST	`connection` = نام نقطه، `url` = مسیر/کوئری (`{Name}` = پارامتر گزارش با `jsonPath`، URL-encode)، مسیر آرایه = `data.items`	`orders?year={Year}` ، `data.items`
`json` ثابت	`json` = متن JSON ثابت (جدول‌های مرجع، ثابت‌ها)	`[{"Code":"A","Title":"Alpha"}]`

بعد نام مجموعه‌ی داده را مثل مجموعه‌ی SQL به‌عنوان منبع نوار (`Orders` ، `Orders.Lines`...) یا منبع Cross-tab بدهید. دکمه‌ی **آزمایش** مجموعه را اجرا می‌کند، تعداد ردیف را نشان می‌دهد و ستون‌های کشف‌شده را در درخت فیلدها می‌گذارد.

ایمنی: فقط http/https؛ آدرس نمی‌تواند از میزبان/پروتکل ثبت‌شده خارج شود؛ سقف پیش‌فرض ۳۰ ثانیه و ۲۰ مگابایت پاسخ (`HttpEndpoint`)؛ در پیش‌نمایش طراح، خطای منبع به هشدار تبدیل و از داده‌ی نمونه استفاده می‌شود.

فیلتر و تابع سفارشی

```
o.AddFunction("Tax", a => Convert.ToDecimal(a[0]) * 0.09m);      // {{Tax(Price)}} و مقدار ذخیره‌شده‌ی =TAX(B2)
o.AddFilter("stars", (value, arg) => new string('*', Convert.ToInt32(value)));   // {{Rating|stars}}
```

---

۱۳) ورود از فایل Excel

دکمه‌ی **xlsx** (یا منوی ویندوزی) فایل `.xlsx` را به قالب تبدیل می‌کند: برگه‌ها، مقدارها، **فرمول‌ها**، فونت/رنگ (شامل رنگ‌های تم)/کادر/تراز/قالب عدد، ادغام، عرض/ارتفاع، فریز، فیلتر، RTL، قالب شرطی، اعتبارسنجی، لینک، **تصویر**، تنظیمات چاپ. سپس سلول‌های داده‌ای را با `{{Field}}` عوض کنید و نوار بگذارید. **وارد نمی‌شود:** نمودار، PivotTable، جدول Excel (سلول ساده می‌آید)، ماکرو. ایمنی: بدون XXE، سقف ۵۰ مگابایت و ۵۰۰هزار سلول. در کد: `XlsxImporter.Import(bytes, "Name").Template`.

---

۱۴) مرجع تنظیمات و API

`KscExcelOptions`

گزینه	پیش‌فرض	توضیح
`TemplateDirectory`	`ExcelTemplates`	پوشه‌ی قالب‌ها (نسبی به ContentRoot/ریشه‌ی سایت)
`Store`	فایل	ذخیره‌ساز سفارشی

گزینه	پیش‌فرض	توضیح
`DesignerTitle`	`KSC Excel Designer`	عنوان
`Language`	`fa`	`fa` (RTL) یا `en`
`ReadOnlyDesigner`	`false`	ذخیره/حذف ممنوع
`AllowAnonymousDesigner`	`false`	بدون آن، خارج از Development لاگین لازم است
`DownloadPath`	`/kxl-download`	مسیر لینک‌های Viewer
`MaxRows` / `MaxCells`	۱,۰۴۸,۵۷۶ / ۳,۰۰۰,۰۰۰	سقف‌ها (MaxCells در Streaming اعمال نمی‌شود)
`PreviewMaxRows`	۵۰۰	سطرهای پیش‌نمایش
`AllowSql`, `AddConnection`, `SqlCommandTimeoutSeconds`, `SqlMaxRows`	`false`, —,۳۰,۱۰۰,۰۰۰	SQL
`AddProvider(name, fn)` / `AddHttpEndpoint(name, baseUrl, headers)`	—	ثبت ارائه‌دهنده‌ی کد / نقطه‌ی REST برای مجموعه‌ی داده‌ی `json` / `provider`
`AddFilter` / `AddFunction`	—	سفارشی‌سازی
`AddModel<T>(name)` / `AddModelFromJson`	—	مدل برای طراح
`UsePdf()` (بسته‌ی Pdf)	—	دکمه‌ی PDF در طراح

API HTTP طراح

پیشوند `{base}/api`؛ نوشتن‌ها هدر `X-KXL-Request: 1` می‌خواهند.

مسیر	کار
`GET /config`	عنوان، زبان، مدل‌ها، قالب‌های شروع، `allowSql`، `pdf`، `connections`
`GET /templates` · `GET/PUT/DELETE /templates/{name}`	فهرست و CRUD
`GET /starters/{key}` · `GET /models/{name}`	مدل / قالب شروع
`POST /validate` · `POST /pdf` · `POST /download` · `POST /preview`	پیش‌نمایش، PDF، xlsx، هشدارها
`POST /import`	ورود `{name, data(base64)}` (xlsx)
`POST /sql/test`	آزمایش کوئری (فقط با `AllowSql`)

قالب JSON

شِمای کامل: `parameters[]` ، `dataSets[]` ، `bands[]` ، `styles{}` ، `sheets[].cells[]` ، `schemaVersion` ، ... قالب‌ها قابل diff و نگهداری در Git هستند. KSC.Excel-Design.md §۵/..

---

۱۵) امنیت

- **طراح می‌تواند قالب بنویسد.** (Core) `RequireAuthorization("پالیسی")` یا `KscExcel.Authorize` (Framework). خارج از Development کاربر ناشناس رد می‌شود مگر `AllowAnonymousDesigner` .
- در Production معمولاً `ReadOnlyDesigner = true` و قالب‌ها از Git/Deploy.
- نوشتن‌ها هدر سفارشی می‌خواهند (محافظت CSRF)؛ نام قالب فقط حرف/عدد/فاصله/`. - _` ؛ `..` رد می‌شود.
- داده‌ی کاربر هرگز **فرمول** نمی‌شود (بدون تزریق فرمول)؛ HTML پیش‌نمایش encode می‌شود؛ لینک‌ها فقط `http/https/mailto` .
- ورود xlsx: بدون XXE، با سقف حجم/سلول. SQL: پیش‌فرض خاموش، فقط SELECT، پارامتری، لاگین فقط‌خواندنی.
- سقف درخواست طراح ۱۶ مگابایت؛ سقف سطر/سلول/صفحه.
- قالب فقط ساختار و `sampleData` دارد؛ **داده‌ی واقعی/حساس** در آن ذخیره نکنید.

---

۱۶) استقرار

1. پوشه‌ی `ExcelTemplates` را به خروجی کپی کنید.
2. چند سرور: قالب‌ها در پوشه‌ی مشترک/دیتابیس (`ITemplateStore`)؛ لینک‌های Viewer در حافظه‌ی همان سرور (Sticky session یا دانلود مستقیم).
3. Docker: پوشه‌ی قالب را Volume کنید؛ برای PDF فونت فارسی نصب کنید.
4. فونت اکسل: فایل فونت را جاسازی نمی‌کند؛ فونت قالب (مثلاً `B Nazanin`) باید روی سیستم کاربر باشد (پیش‌فرض Tahoma).

---

۱۷) کارایی و محدودیت‌ها

سطر × ۶ ستون	زمان	حجم	حافظه
۱٬۰۰۰	≈ ۴۰ ms	۳۷ KB	≈ ۱ MB
۱۰٬۰۰۰	≈ ۰٫۶ s	۳۳۷ KB	≈ ۱۵ MB
۱۰۰٬۰۰۰	≈ ۲٫۸ s	۳٫۳ MB	≈ ۸۰ MB
۱٬۰۰۰٬۰۰۰ (Streaming)	≈ ۱۲٫۶ s	۳۰ MB	≈ ۴۷ MB

- رندر عادی کل گزارش را در حافظه می‌سازد (`MaxCells`)؛ برای بزرگ‌ها Streaming.
- حدود Excel: ۱٬۰۴۸٬۵۷۶ سطر، ۱۶٬۳۸۴ ستون، نام برگه ≤ ۳۱ نویسه.
- رندر هم‌زمان چند درخواست امن است.

---

۱۸) عیب‌یابی و پرسش‌های متداول

مشکل	علت/راه‌حل
طراح ۴۰۱ می‌دهد	خارج از Development لاگین لازم است (یا `AllowAnonymousDesigner`)
ذخیره ۴۰۳ می‌دهد	`ReadOnlyDesigner` فعال است
مقدار سلول خالی است	مسیر فیلد را در پیش‌نمایش بررسی کنید؛ `null` = خالی (`|default:`)
«unknown band» / `#REF!`	نام نوار در `[Band.Field]` دقیق باشد و سلولی در نوار `{{Field}}` داشته باشد؛ برای ستون فرمولی `[Band:E]`. هشدارها در پیش‌نمایش و `Warnings`
جمع ستون صفر	نوار خالی است (عمداً خطا نمی‌دهد)
`The report has more than … rows/cells`	داده را فیلتر کنید یا Streaming یا `MaxCells`
قالب بعد از Publish پیدا نمی‌شود	`ExcelTemplates` را `CopyToOutputDirectory` کنید
PDF فارسی مربع می‌شود (لینوکس)	فونت فارسی نصب کنید / `FontFile`
ستون‌ها راست‌به‌چپ نیست	پنل برگه ← راست‌به‌چپ
SQL اجرا نمی‌شود	`AddConnection`، `AllowSql`، نام اتصال، لاگین فقط‌خواندنی
UI را بدون Build عوض کنم	متغیر `KSC_EXCEL_UI_DIR` = پوشه‌ی `src/KSC.Excel/Designer/UI` (فقط توسعه‌ی خود ابزار)
به Excel روی سرور نیاز است؟	خیر؛ هیچ COM/Interop نیست
`.xls` قدیمی؟	خیر؛ فقط `.xlsx`

---

۱۹) ساخت از سورس، تست و مشارکت

```
src/KSC.Excel/                   هسته (Model, Engine, Writer, Import, Preview, Services, Designer + UI)
src/KSC.Excel.AspNetCore/        آداپتر ASP.NET Core
src/KSC.Excel.Mvc/               آداپتر .NET Framework
src/KSC.Excel.Pdf/               PDF (SkiaSharp + HarfBuzz)
src/KSC.Excel.Designer.Windows/  برنامه‌ی ویندوزی (WPF + WebView2)
samples/KSC.Excel.Sample.Web/    نمونه‌ی اجرایی
tests/KSC.Excel.Tests/           تست‌ها (net10.0 و net48)
tests/KSC.Excel.Pdf.Tests/       تست PDF
docs/                            مستندات
```

```
dotnet build
dotnet test tests/KSC.Excel.Tests          # net10.0 و net48
dotnet test tests/KSC.Excel.Pdf.Tests
```

تست‌ها ساختار فایل xlsx را با `XDocument` بررسی می‌کنند؛ اعتبار نهایی با باز کردن در **Excel واقعی** (COM) انجام شده است. UI بدون `KSC_EXCEL_UI_DIR` :Build. هنگام مشارکت: تغییر رفتار → تست؛ تغییر قالب JSON ناسازگار → افزایش `schemaVersion`؛ مستندات را به‌روز کنید.

---

۲۰) نقشه‌ی راه و محدودیت‌های شناخته‌شده

انجام شده: طراح، موتور، نوار/گروه/Cross-tab، نمودار، پارامتر، SQL، **داده‌ی ارائه‌دهنده/REST/JSON**، عبارت و تابع سفارشی، PDF ،Streaming) **.NET Framework و .NET**)، قالب شرطی، چرخش، سرریز)، **نمایشگر PDF**، **بارکد/QR**، **واترمارک**، **شکست صفحه برای هر گروه**، ورود xlsx، برنامه‌ی ویندوزی (.NET 10 و .NET Framework 4.8)، ویزارد گروه‌بندی، بخش‌های گزارش، راستی‌آزمایی آداپتر MVC 5 روی IIS واقعی.

محدودیت‌ها: 1. فونت جاسازی نمی‌شود (فونت باید روی سیستم کاربر باشد). 2. حافظه در رندر عادی (راه‌حل: Streaming). 3. پیش‌نمایش HTML و PDF تقریب Excelاند (تصویر/بارکد در پیش‌نمایش HTML رسم نمی‌شود). 4. تاریخ شمسی فقط به‌صورت متن (5 .(`|fa` . لینک‌های Viewer در حافظه‌ی همان سرور. 6. LibreOffice/Excel Online/Google Sheets آزمایش نشده (Excel دسکتاپ و IIS Express با .NET Framework 4.8 آزمایش شده). 7. ورود xlsx: نمودار/Pivot/Table وارد نمی‌شود؛ فرمول مشترک با مقدار فعلی. 8. UI طراح برای دسکتاپ بهینه است (لمسی نه).

نیست (به‌دلیل معماری شبکه‌ای موتور): *زیرگزارش به‌صورت قالب جدا* — از نوارهای تو در تو و چند مجموعه‌ی داده استفاده کنید (master/detail/detail را پوشش می‌دهند)؛ اشیای آزاد خارج از شبکه‌ی سلول.

بعدی: کمک‌های OData/GraphQL، ورود نمودار، نسخه‌بندی قالب، انتشار NuGet، Streaming برای جمع بالای جدول، نمودار ترکیبی، بارکدهای EAN-13/Code 39/PDF417.

مجوز: MIT — فایل LICENSE.